

BASIC PROCESSING UNIT**(Single-bus Organisation, Multi-bus Organisation, Hardwired Control, Microprogrammed Control)**Sem: 4th Section (s): COA-CSE-G1, COA-CSCE-G1

Faculty: Prof. Anil Kumar Swain

Lecturer Summary

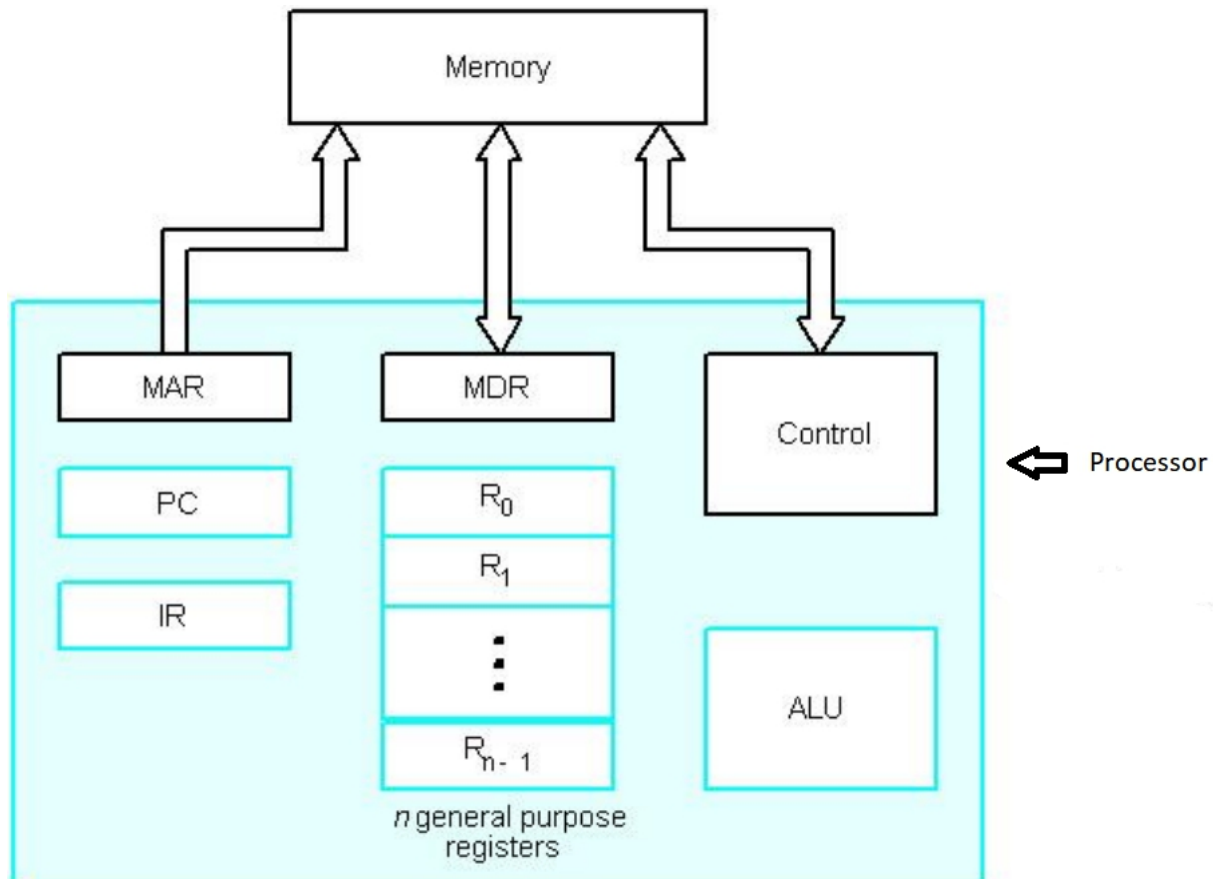
1. Single-bus Organization
2. Multi-bus Organization
3. Hardwired Control
4. Micro-programmed Control

1 Single-Bus Organization

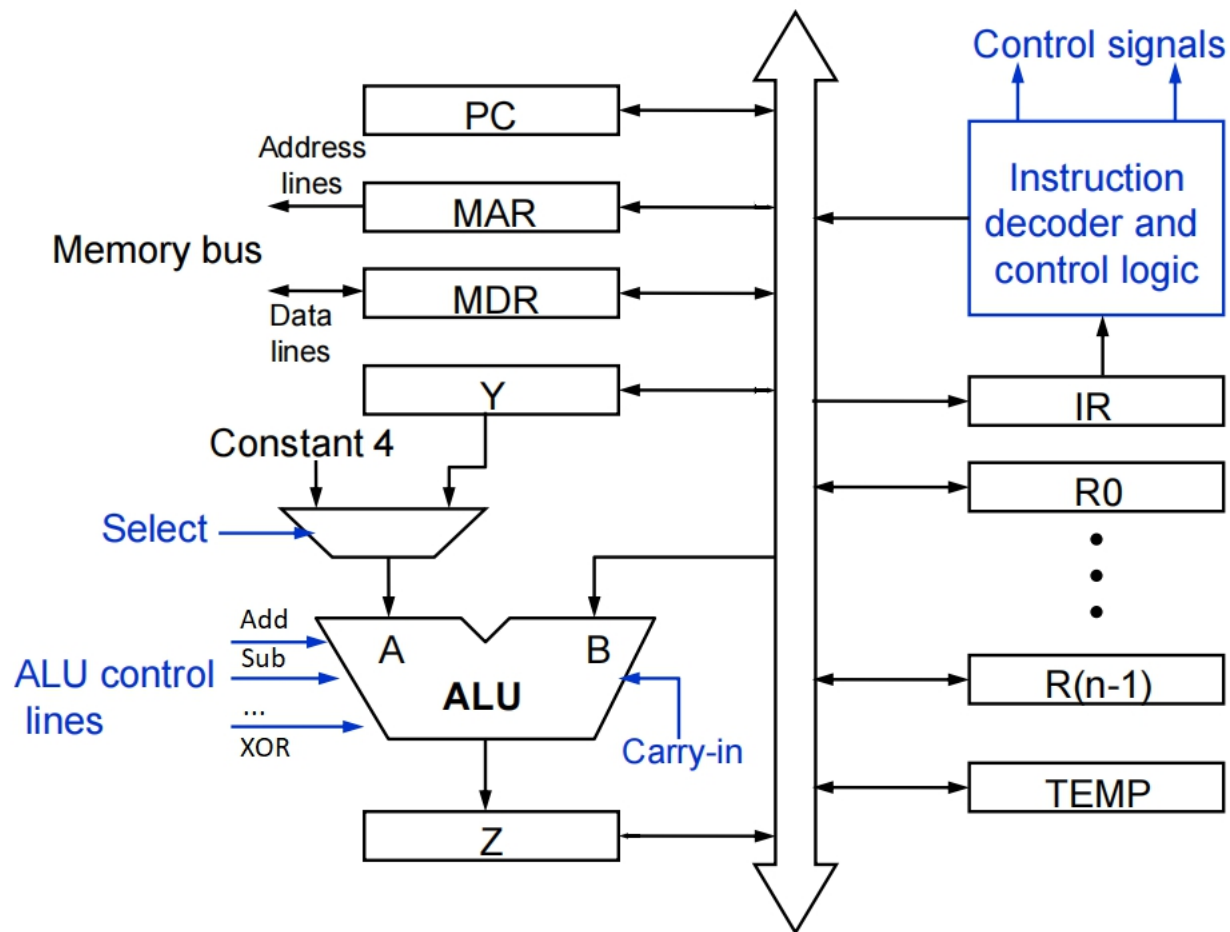
- The unit of the computer that executes machine instructions & coordinates the activities of other units, is called the processing unit, often called Instruction Set processor (ISP) or simply the processor.
- The internal structure of single-bus organization and how it performs the task of fetching, decoding & executing instructions of a program can be examined.

1.1 Fundamental Concepts

- The main building blocks of a processor were discussed. Refer the diagram below:



- The building blocks can be organized and interconnected in a variety of ways. One simple way by using single internal processor bus.



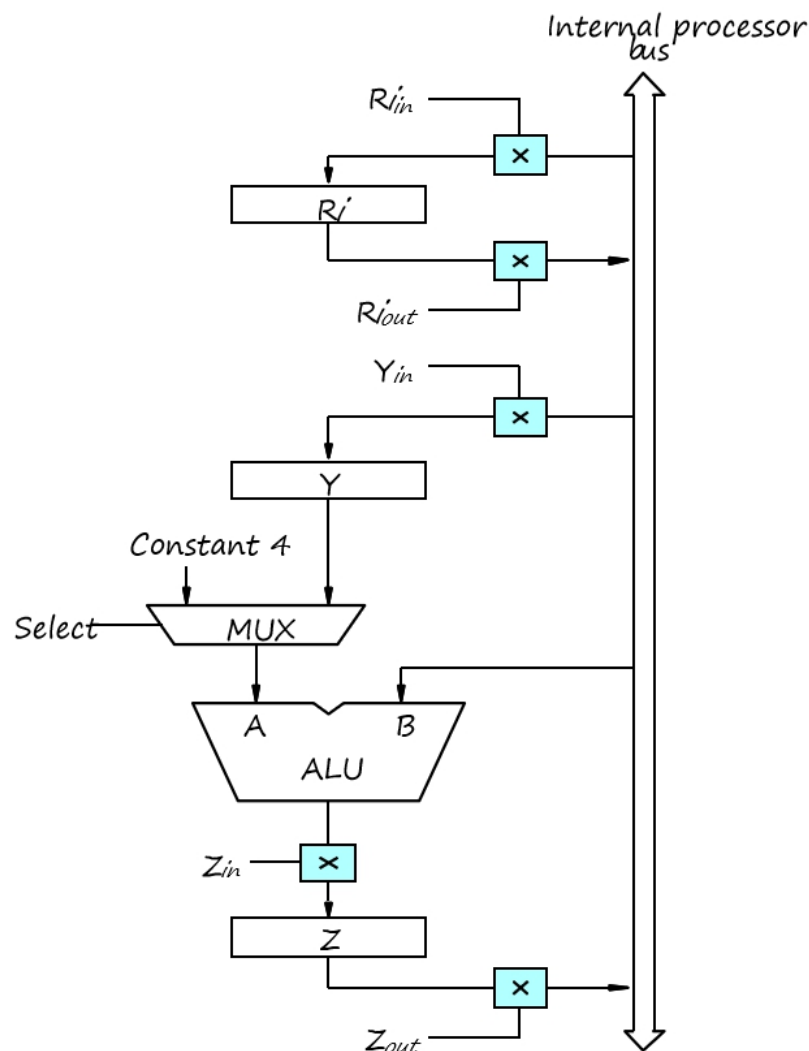
(Single-bus Organisation of the datapath inside a processor)

Source: Ref./CO/5th Ed./Carl Hamacher & Co/Page-413)

- Explanation of the **components of a processor**. The components of the processor are interconnected via a single common bus (in this case).
 - Bus:** A group of lines that serves as a connecting path for several devices is called a bus.
 - PC (Program Counter):** It keeps track of the execution of a program. It contains the memory address of the next instruction to be fetched & executed. In other words, it points to the next instruction that is to be fetched from memory.
 - MAR (Memory Address Register):** It holds the address of the location to be accessed. The contents of MAR is directly connected to the address bus.
 - MDR (Memory Data Register):** It contains the data to be written into or read out of the address location specified by MAR. The contents of MDR are directly connected to the data bus.
 - IR (Instruction register):** An instruction register holds a machine instruction that is currently being executed. In other words, it contains the instruction most recently fetched or executed. The fetched instruction is loaded into an IR, for further analysis of the opcode and operand specifier in the instruction. In other words, instruction registers used to hold a copy of prefetched instructions in the part of the program currently being executed.
 - Instruction decoder & Control logic unit:** It is responsible for implementing the actions specified by the instruction loaded in the IR register.

- g) **ALU (Arithmetic Logic Unit):** An arithmetic logic unit(ALU) is a major component of the processing unit of the a computer system. It does all processes related to arithmetic and logic operations that need to be done on instruction words.
- h) **MUX (Multiplexer):** In electronics, a multiplexer (or mux, also known as a data selector), is a device that selects between several analog or digital input signals and forwards the selected input to a single output line. A multiplexer of 2^n inputs has n select lines, which are used to select which input line to send to the output. In this case 2:1 MUX is used with two possible values of the MUX control input Select as Select 4 & Select Y for selecting constant 4 or register Y respectively.
- i) **General Purpose Registers (R_0 to R_{n-1}):** These registers are used by the programmer for general purpose. The number & use of these processor registers vary considerably from one processor to another.
- j) **Special Temporary Storage Registers (Y, Z & TEMP):** These registers are used by the processor for temporary storage during execution of some instructions. These registers are never used for storing data generated by one instruction fro later use by another instruction.

- **Register Transfer**



(Input and output gating for the registers in Single-bus organization)

Source: Ref./CO/5th Ed./Carl Hamacher & Co/Page-416)

- **Instruction Execution :** To execute an instruction, the processor has to perform the following three steps:
 - **Step 1:** Fetch the contents of the memory location pointed by the PC. The contents of this location are interpreted as an instruction to be executed. Hence, they are loaded into the IR. Symbolically, this can be written as $IR \leftarrow [PC]$
 - **Step 2:** Assuming that the memory is byte addressable & each word length is 32 bits, So increment the contents of the PC by 4, that is, $PC \leftarrow [PC] + 4$
 - **Step 3:** carry out the actions specified by the instruction in the IR
 - Fetch phase:** Step-1 and Step-2 are usually referred as phase
 - Execution phase:** Step-3 is called the execution phase.
- N.B:** An instruction can be executed by performing one or more of the following operations in some specified sequence
- ❖ Transfer a word of data from one processor register to another or to the ALU.
 - ❖ Perform an arithmetic or a logic operation and store the result in a processor register.
 - ❖ Fetch the contents of a given memory location and load them into a processor register.
 - ❖ Store a word of data from a processor register into a given memory location

1.2 Execution of a Complete Instruction

Example-1:

- Write the sequence of control steps required for the single bus structure for the following instruction.

ADD R3, R1

Which adds the contents of R3 register to register R1 and the result is stored in Register R1.

Solution

- Executing this instruction requires the following actions:
 - a) Fetch the instruction
 - b) Perform the addition
 - c) Load the result into R1
- Control sequence for execution of the instruction ADD (R3), R1

Step	Action	
1.	$PC_{out}, MAR_{in}, Read, Select\ 4, ADD, Z_{in}$	}
2.	$Z_{out}, PC_{in}, Y_{in}, WMFC$	
3.	MDR_{out}, IR_{in}	
4.	$R1_{out}, Y_{in}$	}
6.	$R3_{out}, Select\ Y, ADD, Z_{in}$	
7.	Z_{out}, R_{in}, End	

Example-2:

- Write the sequence of control steps required for the single bus structure for the following instruction.

ADD (R3), R1

Which adds the contents of memory location pointed by R3 register to register R1 and the result is stored in Register R1.

Solution

- Executing this instruction requires the following actions:
 - Fetch the instruction
 - Fetch the first operand
 - Perform the addition
 - Load the result into R1
- Control sequence for execution of the instruction ADD (R3), R1

Step	Action	
1.	PC _{out} , MAR _{in} , Read, Select 4, ADD, Z _{in}	}
2.	Z _{out} , PC _{in} , Y _{in} , WMFC	
3.	MDR _{out} , IR _{in}	
4.	R3 _{out} , MAR _{in} , Read	}
5.	R1 _{out} , Y _{in} , WMFC	
6.	MDR _{out} , Select Y, ADD, Z _{in}	
7.	Z _{out} , R _{in} , End	

- Explanation**

- ✓ As instructions are store in main memory and that need to transfer to processor first for the execution of it, so first three control steps are same for any generic assemble language.
- ✓ Processor has the inromation about the starting address of the program stored in main memory that is the content of PC. From here start writing the comntrol instructions.
- ✓ Once the instruction is fetched from main memory, then look for what operation is to be performed.
- ✓ For this opeartion make available of the operands with ALU inputs by writing appropriate control instructions.
- ✓ If both the operands are directly stored in registers, then they are internal to the processor, so by using registers in & out command with common bus, operands can be easily available with the ALU inputs.
- ✓ If any or both the operands are stored in main memory and its locaion is available with the processor register,

Example-3:

- Write the sequence of control steps required for the single bus structure for the following instruction.

ADD 12(R3), R1

Which adds the contents of memory location pointed by ([R3]+12) to register R1 and the result is stored in Register R1.

Solution

Step	Action	
1.	PC _{out} , MAR _{in} , Read, Select 4, ADD, Z _{in}	}
2.	Z _{out} , PC _{in} , Y _{in} , WMFC	
3.	MDR _{out} , IR _{in}	
4.	Offset field of IR _{out} , Y _{in}	}
5.	R3 _{out} , Select Y, Add, Z _{in}	
6.	Z _{out} , MAR _{in} , Read	
7.	R1 _{out} , Y _{in} , WMFC	
8.	MDR _{out} , Select Y, ADD, Z _{in}	
9.	Z _{out} , R1 _{in} , end	

Example-4:

- Write the sequence of control steps required for the single bus structure for the following instruction

ADD -(R3), R1

Which adds the contents of memory location pointed by ([R3]-4) to register R1 and the result is stored in Register R1.

Solution

Step	Action	
1.	PC _{out} , MAR _{in} , Read, Select 4, ADD, Z _{in}	}
2.	Z _{out} , PC _{in} , Y _{in} , WMFC	
3.	MDR _{out} , IR _{in}	
4.	R3 _{out} , Select4, Sub, Z _{in}	}
5.	Z _{out} , MAR _{in} , Read	
6.	R1 _{out} , Y _{in} , WMFC	
7.	MDR _{out} , Select Y, ADD, Z _{in}	
9.	Z _{out} , R1 _{in} , end	

Example-5:

- Write the sequence of control steps required for the single bus structure for the following instruction

ADD (R3)-, R1

Which adds the contents of memory location pointed by R3 register to register R1 and the result is stored in Register R1.

Solution

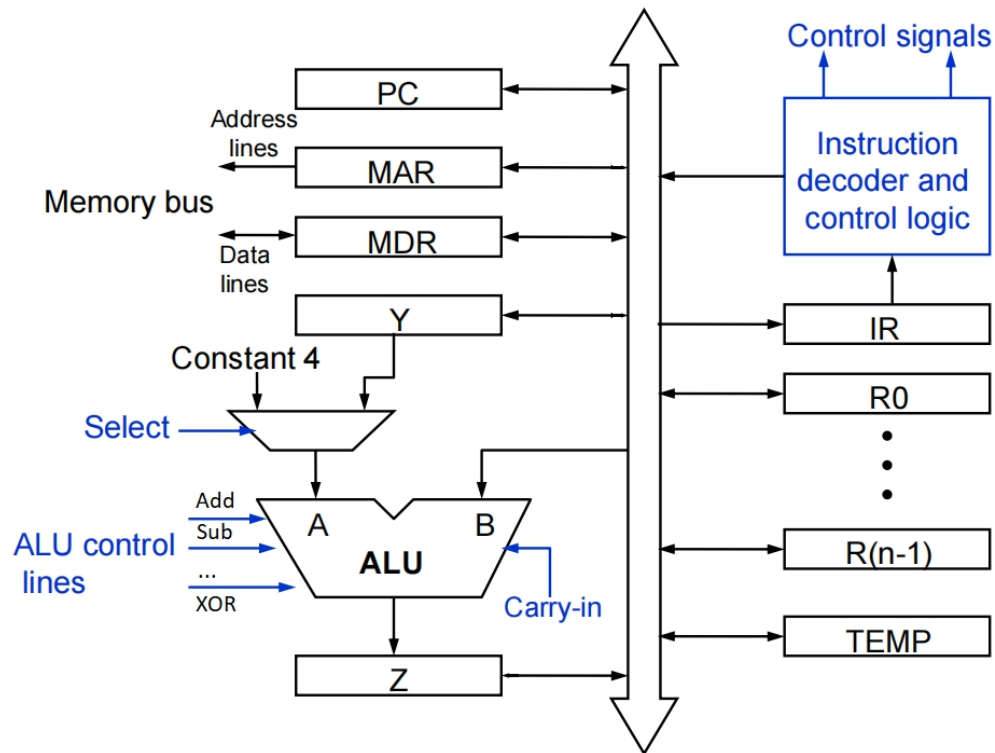
N.B: If the instruction **ADD (R3)-, R1** is a valid generic assemble instruction & post auto decrement mode is allowed, then the **sequence of control steps required for the single bus structure** are as follows:

Step-1 to 3 to fetch the instruction from memory to processor is same as above.

(As it is post decrement, so first the address pointed by register R3 will be used for operand fetch, then the content of register R3 will be decremented by 4.)

Step-4. R3_{out}, MAR_{in}, Read, Select 4, SUB, Z_{in}

Step-5. Zout, R3in
 Step-6. R1out, Yin, WMFC
 Step-7. MDRout, Select Y, ADD, Zin
 Step-8. Zout, Rin, End



BRANCH INSTRUCTIONS

Example-6:

- Write the sequence of control steps required for the single bus structure for the following instruction

Jump L1

Which unconditionally transfers the controls to location L1 for further execution of instructions.

Solution

Step	Action	
1.	PC _{out} , MAR _{in} , Read, Select 4, ADD, Z _{in}	}
2.	Z _{out} , PC _{in} , Y _{in} , WMFC	
3.	MDR _{out} , IR _{in}	
4.	Offset-field-of-IR _{out} , Select Y, ADD, Z _{in}	}
5.	Z _{out} , PC _{in} , End	

Example-6:

- Write the sequence of control steps required for the single bus structure for the following instruction

Branch <0 Label1

Which transfers the controls to location Label1 for further execution of instructions if the condition satisfied by Branch<0.

Solution

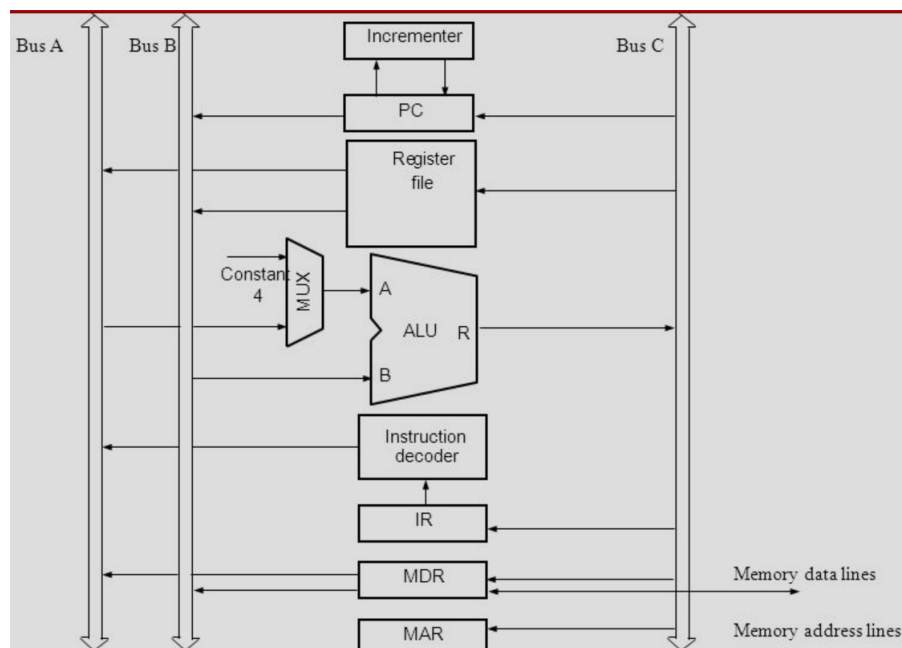
Step	Action	
1.	PC _{out} , MAR _{in} , Read, Select 4, ADD, Z _{in}	}
2.	Z _{out} , PC _{in} , Y _{in} , WMFC	
3.	MDR _{out} , IR _{in}	
4.	Offset-field-of-IR _{out} , Select Y, ADD, Z _{in} if N=0, then End	}
5.	Z _{out} , PC _{in} , End	

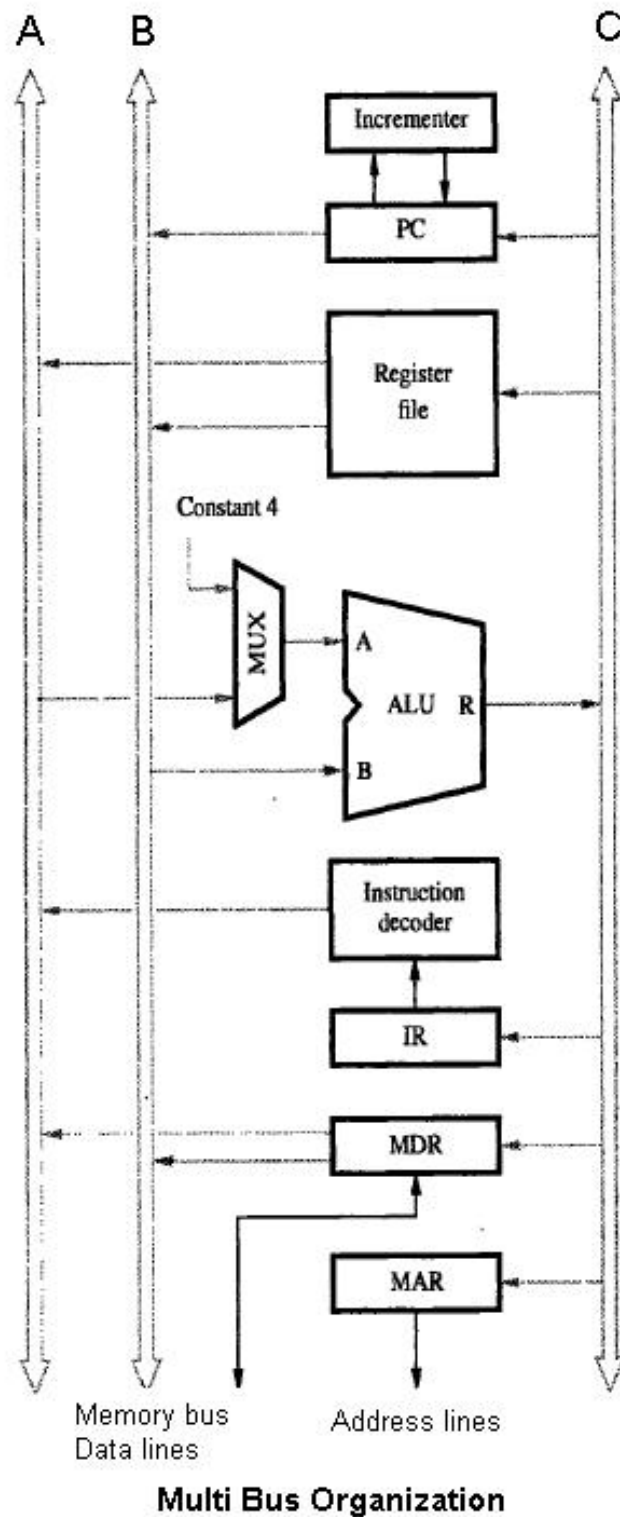
2 Multi-bus Organization (Three-bus Organization)

- In case of single-bus structure, the resulting control sequences for an instruction are quite long, because only one data item can be transferred over the bus in a clock cycle.
- To reduce the number of steps needed, most commercial processors provide multiple internal paths that enable several transfers to take place in parallel.
- Here the internal structure of three-bus organization and how it performs the task of fetching, decoding & executing instructions of a program can be examined.

2.1 Block Diagram of Three-bus organization

- The main building blocks of a processor were discussed. Refer the diagram below:





Example-7:

- Write the sequence of control steps required for the three bus structure for the following instruction

Add R4, R5, R6

Which performs the addition of register R4 to R5 and the result is stored in R6.

Solution

Step	Action	
1.	PCout, R=B, MARin, Read, IncPC	} Fetch instruction
2.	WMFC	
3.	MDRoutB, R=B, IRin	
4.	R4outA, R5outB, SelectA, Add, R6in, End	Execute the instruction

Example-7:

- Write the sequence of control steps required for the three bus structure for the following instruction

Move (R1), R2

Which performs the addition of register R4 to R5 and the result is stored in R6.

Solution

Step	Action	
1.	PCout, R=B, MARin, Read, IncPC	} Fetch instruction
2.	WMFC	
3.	MDRoutB, R=B, IRin	
4.	R1outB, R=B, MARin, Read	} Execute the instruction
5.	WMFC	
6.	MDRoutB, R=B, R2in, End	

Example-8:

- Write the sequence of control steps required for the three bus structure for the following instruction

Add (R1), R2

Which performs the addition of register R4 to R5 and the result is stored in R6.

Solution

Step	Action	
1.	PCout, R=B, MARin, Read, IncPC	} Fetch instruction
2.	WMFC	
3.	MDRoutB, R=B, IRin	
4.	R1outB, R=B, MARin, Read	} Execute the instruction
5.	WMFC	
6.	MDRoutB, R2outA, Select A, Add, R2in, End	